

Find `htmlSelector` for Synthetic Interactions

This document provides a comprehensive guide on how to find the `htmlSelector` necessary for configuring synthetic interactions such as clicks and hovers. The `htmlSelector` is essential for specifying which elements on a webpage should be targeted during the pre-rendering process.

What is `htmlSelector`?

The `htmlSelector` is a string used to uniquely identify elements on a webpage based on their HTML structure. It is critical for performing synthetic interactions effectively and accurately, ensuring the correct elements are targeted during the automation of interactions in prerendering.

DOM and Shadow DOM

The `htmlSelector` is contained within the DOM or the shadow DOM of the web page.

- **Document Object Model (DOM):** This is the standard model that describes how all elements in a web page are arranged. The DOM represents the document as a tree of objects, which can be manipulated with languages like JavaScript.
- **Shadow DOM:** This part of the web standards allows developers to create their own encapsulated HTML tags, styles, and scripts, without worrying about them conflicting with other parts of the web page. It effectively shields a part of the page's HTML from being affected by the main document's CSS and JavaScript. This is particularly useful in complex web applications and components like web components, where you want to keep the component's internal functionality separate and hidden from the rest of the application.

Special Consideration for Shadow DOM Elements

For elements encapsulated within a shadow DOM, prepend the selector with `>>>` to ensure Prerendering can accurately target and interact with these elements. The `>>>` notation allows the selector to "pierce" through the shadow boundary, enabling interaction with elements that are otherwise encapsulated.

Find the `htmlSelector`

Finding HTML selectors effectively is a crucial skill for web developers, especially when you need to manipulate or access specific elements on a web page. Here's a step-by-step guide to finding HTML selectors.

1. Choose Your Browser and Open Developer Tools

First, open the Developer Tools in your web browser. You can usually do this by right-clicking on an element in the page and then selecting **Inspect** or by using keyboard shortcuts:

- For **Chrome, Edge, and Firefox**: Press `Ctrl+Shift+I` (Windows/Linux) or `Cmd+Option+I` (Mac).
- For **Safari**: First, enable the Developer menu from Safari's preferences (under the `Advanced` tab), then press `Cmd+Option+I`.

2. Inspect the Element

With the Developer Tools open, use the `Elements` tab (named `Inspector` in Firefox) to view the HTML structure of the page. You can hover over different parts of the HTML code, and the corresponding parts of the web page will be highlighted.

3. Identify and Copy the Selector

There are several considerations for choosing the selector.

Use the ID

If the element has an `id` attribute, this can serve as a straightforward selector. Example: `#login-button`.

Use a Complex Selector

For elements without an `id`, or when a more specific path is needed, use the right-click context menu over the highlighted HTML code in the Developer Tools and select `Copy > Copy selector`. This gives you a string that uniquely identifies the element in CSS terms. This might be a combination of element types, classes, IDs, and pseudo-classes.

For Shadow DOM Elements

If the element is within a shadow DOM, use `>>>` followed by the specific selector. For example, `>>> #__next > div > header > div.chakra-container.css-1ichkct > div > div.chakra-stack.css-jojt9m > a.chakra-link.css-i0nad5`.

- You might need to use JavaScript to access the `shadowRoot` of the custom element. For example: `document.querySelector('custom-element').shadowRoot.querySelector('element')`.
- Some browsers' Developer Tools allow you to inspect elements within Shadow DOM directly. Ensure that the option to show user agent shadow DOM is enabled in the settings of the Developer Tools.

Consider Specificity and Efficiency

The automatically generated selector might not always be the most efficient or simplest. It might be overly specific. Simplify the selector if possible, keeping only the parts that are necessary to uniquely identify the element.

For example, if the generated selector is `html > body > div:nth-child(3) > div > p > a`, and you know the `div` element has the id `login-container`, you might simplify this to `#login-container > p > a`.

Testing the `htmlSelector`

Once you have identified a potential `htmlSelector`, it's important to test its functionality:

1. Open the Console Tab in Developer Tools

Within the Developer Tools, find and click on the `Console` tab. This is where you can run JavaScript code directly against the page you are viewing.

2. Test Your Selector

In the console, you can use the `document.querySelector()` method to test your selector. This method returns the first element within the document that matches the specified selector, or `null` if there are no matches. Here's how to use it:

- Type `document.querySelector("yourSelector")` into the console, replacing `"yourSelector"` with the CSS selector you want to test.
- Press `Enter` to run the command.

Example

If you want to test a selector for an element with the ID `login-button`, you would type:

```
document.querySelector("#login-button")
```

If the selector is correct and the element exists, the console will display the HTML of that element. If the selector is incorrect or no element matches, it will return `null`.

3. Analyze the Results

- If the element appears in the console, inspect the output to ensure it's the correct element you intended to select.

- If `null` is returned, double-check your selector for any errors, such as typos, incorrect classes, or IDs. You may also need to ensure that the element is not dynamically loaded after some interactions, which might require waiting for certain events before the element can be targeted.

4. Adjust as Necessary

If the initial test doesn't return the correct element, then modify your selector and test again. This iterative process helps refine your selector to be both accurate and efficient.

Best Practices

These best practices help ensure that the selectors used in prerendering scenarios are reliable, maintainable, and effective in driving the intended synthetic interactions.

Prioritize Stability in Selectors

Choose selectors that are less likely to change. Avoid using dynamically generated classes or IDs that might vary between sessions or deployments. Opt for selectors based on stable structural elements or attributes that are consistent across different states of the application.

Use Data Attributes

Consider using custom data attributes (for example, `data-test-id="submit-button"`) for critical elements that are interacted with during synthetic interactions. This approach isolates the selector from style and structural changes, making your tests more resilient to changes in the design.

Minimize Dependency on DOM Structure

Avoid selectors that are overly dependent on the DOM hierarchy, such as descendant selectors that traverse multiple layers (such as `div > ul > li > a`). These can break if there are changes in the structure of the DOM. Instead, use direct child selectors (such as `div > a`) or even better, selectors that target elements directly.

Incorporate Robustness to Dynamic Content

For elements that might appear as part of dynamic content loading, ensure your selectors are robust. This might involve waiting for elements to become visible or checking for the presence of an element before interacting with it in your scripts.

Explicit State Selectors

For interactions like hover or focus, explicitly define selectors that mimic these states. For example, using `:hover` or `:focus` within your CSS or when scripting interactions to ensure that the state is considered during the prerendering process.

Testing and Validation

Regularly test your selectors as part of your prerendering process. Ensure they not only find the elements but also trigger the intended interactions correctly across all scenarios and edge cases.

Document Selector Intent

Clearly document what each selector is targeting and why it's chosen, especially for those involved in the synthetic interactions. This documentation is invaluable for maintenance and when modifications are needed as the application evolves.

Cross-Browser Compatibility

Verify that your selectors and their synthetic interactions work consistently across all targeted browsers. This ensures that the prerendered content behaves as expected, regardless of the user's browser.

Feedback Loops

Implement mechanisms to capture and review the success rate of interactions triggered by your selectors. Use this feedback to refine and improve the selector strategy continuously.

Accessibility Considerations

Ensure that the elements targeted for synthetic interactions are also accessible. This not only improves the quality of your prerendered pages but also ensures compliance with web accessibility standards.

 [Edit this page](#)